

# Desde cero hasta ChatGPT

Día 4: Decoder-only y Agentes

José Antonio Hernández López

4 de marzo de 2026

# Roadmap general

1. Redes neuronales y Pytorch - **Día 1**
2. Redes recurrentes y tokenización - **Día 2**
3. Mecanismo de atención, Transformers, Pre-training y Fine-tuning - **Día 3**
4. Modelos decoder-only. De GPT a ChatGPT - **Día 4.1** ←
5. Agentes LLM - **Día 4.2** ←

# Roadmap del día 4

1. Modelos Decoder-only
2. De GPT-3 a ChatGPT
3. Parameter efficient fine-tuning (PEFT)
4. Cuantización
5. Librerías para inferencia
  - Librerías open-source para ejecución local
  - APIs comerciales
6. Agentes LLM
7. Patrones *agénticos*
  - ReAct
  - CodeAct
  - Planning first
8. Librerías de agentes
  - Smolagents
  - LangChain
9. Model Context Protocol

Modelos Decoder-only

# Origen del Transformer

## Contexto histórico

El **Transformer** surge originalmente como una arquitectura **encoder–decoder** para tareas de **traducción automática**.

- ▶ Propuesto en 2017 en *“Attention is All You Need”*
- ▶ Diseñado para mapear:

secuencia de entrada → secuencia de salida

## Idea

El Transformer completo = Encoder + Decoder

Ya hemos visto lo que se puede hacer con el Encoder... nos queda el Decoder

## Factor clave

En traducción de texto, se aproxima la siguiente distribución:

$$P(x_t | x_1, \dots, x_{t-1}, y).$$

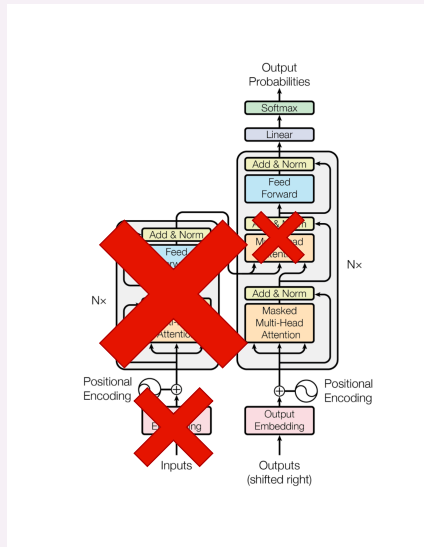
Donde  $x_i$  son los tokens de la traducción a predecir e  $y$  es la frase a traducir.

Si quitamos el encoder y sólo dejamos el decoder, jugando un poco con las atenciones nos queda esto:

$$P(x_t | x_1, \dots, x_{t-1}).$$

Es decir, un modelo del lenguaje. El decoder se entrena de manera autoregresiva para predecir el siguiente token dados los anteriores.

# Factor clave

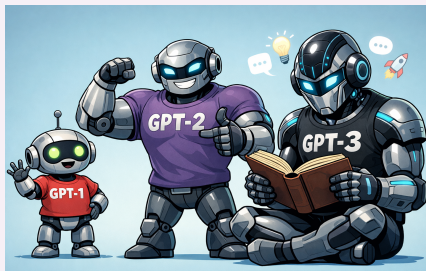


# Modelos GPT

## ¿Qué son los GPTs?

Los **GPT (Generative Pre-trained Transformers)** son una familia de **modelos de lenguaje generativos** basados en la arquitectura transformer, diseñados para aprender a partir de grandes cantidades de texto.

- ▶ **Generative:** producen texto de forma autoregresiva
- ▶ **Pre-trained:** se entrenan primero de manera no supervisada
- ▶ **Transformer:** usan el transformer
- ▶ Propuestos y desarrollados por **OpenAI**



# GPT-1 (2018): Pre-training y fine-tuning

## ¿Qué se descubrió?

Similar a BERT, se descubrió que el decoder-only pre-entrenado en predecir el siguiente token era de bastante utilidad si se fine-tuneaba.

- ▶ Paradigma **Pre-training** → **Fine-tuning**
- ▶ Similar a BERT, se añadía una cabeza lineal para la clasificación
- ▶ BERT es mejor para las tareas de clasificación
- ▶ Modelo 117 millones de parámetros
- ▶ 4 GBs de texto

## GPT-2 (2019): La cosa se pone interesante

### ¿Qué se descubrió?

Que **escalar el modelo y los datos**, sin cambiar la arquitectura ni el pre-entrenamiento, produce **capacidades emergentes**.

- ▶ Modelo mucho más grande (varias versiones, la más grande 1542M de parámetros)
- ▶ 40 GBs de texto
- ▶ El modelo realiza tareas *no entrenadas explícitamente*

# GPT-2 (2019): La cosa se pone interesante

## Context (passage and previous question/answer pairs)

Tom goes everywhere with Catherine Green, a 54-year-old secretary. He moves around her office at work and goes shopping with her. "Most people don't seem to mind Tom," says Catherine, who thinks he is wonderful. "He's my fourth child," she says. She may think of him and treat him that way as her son. He moves around buying his food, paying his health bills and his taxes, but in fact Tom is a dog.

Catherine and Tom live in Sweden, a country where everyone is expected to lead an orderly life according to rules laid down by the government, which also provides a high level of care for its people. This level of care costs money.

People in Sweden pay taxes on everything, so aren't surprised to find that owning a dog means more taxes. Some people are paying as much as 500 Swedish kronor in taxes a year for the right to keep their dog, which is spent by the government on dog hospitals and sometimes medical treatment for a dog that falls ill. However, most such treatment is expensive, so owners often decide to offer health and even life insurance for their dog.

In Sweden dog owners must pay for any damage their dog does. A Swedish Kennel Club official explains what this means: if your dog runs out on the road and gets hit by a passing car, you, as the owner, have to pay for any damage done to the car, even if your dog has been killed in the accident.

Q: How old is Catherine?

A: 54

Q: where does she live?

A:

**Model answer:** Stockholm

**Turker answers:** Sweden, Sweden, in Sweden, Sweden

Figura: Ejemplo extraído de paper original

## GPT-3 (2020): Consolidación

### ¿Qué se descubrió?

Que un modelo suficientemente grande puede **aprender tareas a partir del prompt**, sin modificar sus parámetros.

- ▶ Se consolida el **in-context learning** con todas sus técnicas asociadas: zero-shot, few-shot, etc.
- ▶ Surge el **prompt engineering**
- ▶ Tamaño del modelo: 175 billones de parámetros
- ▶ 570 GBs de texto

# GPT-3 (2020): Consolidación

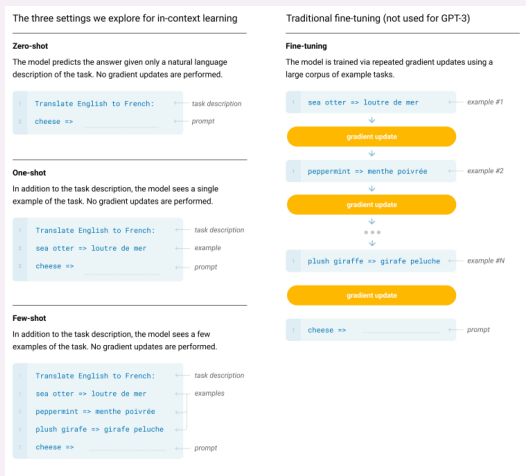


Figura: Ejemplo extraído de paper original

# Large Language Models

- ▶ A partir de GPT-3, a todos estos modelos que han sido entrenados con muchos datos en tareas relacionadas con el modelado del lenguaje se les empiezan a llamar **Large Language Models** (LLMs):
  - ▶ **Large**: red neuronal profunda con millones o billones de parámetros
  - ▶ **Language Models**: modelo entrenado en una tarea de modelado del lenguaje (autoregresiva o similar)

De GPT-3 a ChatGPT

## Limitaciones de GPT-3

Es cierto que GPT-3 presenta propiedades de in-context learning emergentes. Sin embargo:

- ▶ GPT-3 ha sido entrenado **únicamente** para generar texto probable
- ▶ No está optimizado para seguir instrucciones

Esto provoca:

- ▶ Hay que trabajarse muy bien el prompt
- ▶ Modelo poco robusto.

Ejemplo de prompt: *Resume el siguiente texto en 3 líneas*

*<texto>*

- ▶ A veces resume
- ▶ A veces continúa el texto
- ▶ A veces ignora la instrucción
- ▶ A veces no para de generar texto

# Foundation Model

Este problema no es solo de GPT-3. Es un problema general de los foundation models:

## Foundation Model

Un foundation model es un LLM entrenado sobre grandes volúmenes de datos textual de manera autoregresiva (predecir el siguiente token).

- ▶ Ejemplos:
  - ▶ GPT-3
  - ▶ <https://huggingface.co/Qwen/Qwen3-8B-Base>
  - ▶ <https://huggingface.co/BSC-LT/ALIA-40b>
  - ▶ <https://huggingface.co/deepseek-ai/DeepSeek-V3.1-Base>
  - ▶ etc.
- ▶ Si el nombre del model acaba con `-base`, suele ser indicador de que es un foundation model
- ▶ Se llama *foundation* porque LLMs posteriores se construyen sobre él

Ejemplo en el notebook/colab.

# Supervised Fine-Tuning

## Objetivo

Enseñar al modelo a **seguir instrucciones humanas** de forma directa.

¿Cómo se hace?

- ▶ Se parte de un dataset etiquetado por humanos (e.g., conversaciones o instrucción/respuesta)
- ▶ De manera similar al pre-training, el modelo aprende a predecir la respuesta dada una instrucción. El objetivo es también  $P(x_t|x_1, \dots, x_{t-1})$
- ▶ Dependiendo del dataset usado, el modelo resultante puede soportar conversaciones *single-turn* o *multi-turn* (instrucción/respuesta y diálogo)

## Resultado

Se obtiene un asistente capaz de seguir instrucciones (*single-turn* o *multi-turn*).

# Ejemplo: Salamandra

## Modelo

**Salamandra** es un modelo que soporta conversaciones obtenido aplicando **Supervised Fine-Tuning** sobre un modelo base.

- ▶ Parte de ALIA  
<https://langtech-bsc.gitbook.io/alia-kit/modelos/modelos-de-texto>
- ▶ Entrenado por el equipo NLP del Barcelona Supercomputing Center
- ▶ Idiomas: inglés, español y catalán
- ▶ Se parte de un modelo base (2b y 7b de parámetros) "fuerte" en español y catalán

```
User:      <start_of_turn>user  
           Toc toc<end_of_turn>  
           <start_of_turn>assistant  
Assistant: ¿Quién es?<end_of_turn>  
User:      <start_of_turn>user  
           Talandas<end_of_turn>  
           <start_of_turn>assistant  
Assistant: ¿Qué Talandas?<end_of_turn>
```

Figura: Prompt usado en Salamandra, extraído del paper original.

# Ejemplo: WizardCoder

## Modelo

**WizardCoder** es un modelo de generación de código obtenido aplicando **Supervised Fine-Tuning** sobre un modelo base.

- ▶ Se parte de StarCoder 15B y CodeLlama-34B-Python
- ▶ Dataset SFT. Pares sintéticos (*instrucción de programación, respuesta*) de alta calidad
- ▶ Tipo de entrenamiento: predicción de la respuesta dada una instrucción. No es un chatbot, solo instrucciones *single-turn*

### Prompt for Fine-Tuning Format

Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.

### Instruction:  
{instruction}

### Response:

**Figura:** Prompt usado en WizardCoder, extraído del paper original.

## Takeaways

- ▶ Aunque un foundation model (por ejemplo, GPT-3) es potente, no sigue de manera robusta las instrucciones del usuario
- ▶ Para obtener un asistente, es necesario pasar por un proceso de SFT
- ▶ Las ideas principales de SFT son:
  - ▶ Usar un dataset de instrucciones/respuesta o de chats
  - ▶ Entrenar el modelo en predecir el siguiente token (igual que en el pre-training)
- ▶ Es importante leer la documentación de cómo ha sido SFT y cómo se utiliza. Por ejemplo, WizardCoder solo funciona usando ese prompt y no soporta *multi-turn* chats

### Conclusión

Después del SFT, se obtiene un asistente capaz de seguir instrucciones humanas.

Demostración en Colab con Salesforce/codegen-350M-mono

## Limitación de SFT

El modelo SFT puede seguir instrucciones pero puede generar respuestas que no son humanas o no siguen preferencias del usuario.

- ▶ Ejemplo:

*Escribe código que ordene un array.*

- ▶ Respuesta.

*Aquí tienes:*

```
def sort_array(a):  
    return sorted(a)
```

## Limitación de SFT

El modelo SFT puede seguir instrucciones pero puede generar respuestas que no son humanas o no siguen preferencias del usuario.

- ▶ Ejemplo:

*Escribe código que ordene un array.*

- ▶ Respuesta.

*Aquí tienes:*

```
def sort_array(a):  
    return sorted(a)
```

Es correcto pero... no merece el usuario algo más de explicación?

- ▶ Respuesta.

*Vamos a usar la función built-in de Python que ordena el array de manera ascendente.*

```
def sort_array(a):  
    """  
        Ordena una lista de números en orden ascendente.  
    """  
    return sorted(a)
```

- ▶ La segunda respuesta es mejor que la primera ya que es más explicativa

## Limitación de SFT

El modelo SFT puede responder correctamente a una instrucción, pero no detectar ambigüedad ni pedir aclaraciones cuando un humano lo haría.

- ▶ Ejemplo:

*Traduce esta frase.*

*I saw her duck.*

- ▶ Respuesta.

*Vi su pato.*

## Limitación de SFT

El modelo SFT puede responder correctamente a una instrucción, pero no detectar ambigüedad ni pedir aclaraciones cuando un humano lo haría.

- ▶ Ejemplo:

*Traduce esta frase.*

*I saw her duck.*

- ▶ Respuesta.

*Vi su pato.*

Es correcto pero... ¿es realmente lo que el usuario quería?

- ▶ Respuesta.

*La frase es ambigua. Puede significar:*

- ▶ *Vi su pato* (el animal)

- ▶ *La vi agacharse* (verbo *to duck*)

*¿Podrías dar más contexto para elegir la traducción correcta?*

- ▶ La segunda respuesta es mejor que la primera ya que:

- ▶ Detecta ambigüedad semántica

- ▶ Se comporta como lo haría un humano

# Reinforcement learning

## Objetivo

Enseñar al modelo a **generar respuestas humanas o más alineadas con las preferencias humanas.**

¿Cómo se hace?

- ▶ Se parte de un dataset etiquetado por humanos. Usualmente, el dataset contiene pares (*instrucción, lista de respuestas ordenadas por calidad*)
- ▶ Se utiliza un algoritmo de **reinforcement learning** para ajustar el modelo y asignar mayor probabilidad a las respuestas preferidas
- ▶ Técnicas más usadas:
  - ▶ **RLHF (Reinforcement Learning from Human Feedback)**
  - ▶ DPO (Direct Preference Optimization)

## Resultado

El modelo no solo responde correctamente, sino que aprende a **responder mejor**: con mayor utilidad, claridad, tono adecuado y alineación con expectativas humanas.

# RLHF: Reinforcement Learning from Human Feedback

## Pipeline típico

1. **Modelo base** (pre-entrenado + SFT)
2. **Recogida de feedback humano:**
  - ▶ Para una instrucción, el modelo genera varias respuestas
  - ▶ Humanos ordenan las respuestas por calidad
3. **Entrenamiento de un Reward Model (RM):** aprende a predecir qué respuestas prefieren los humanos
4. **Optimización por RL:**
  - ▶ El LLM se ajusta para maximizar la recompensa del RM sin desviarse mucho del modelo base
  - ▶ Algoritmo típico: PPO (Proximal Policy Optimization)

# Ejemplo: ChatGPT

## Modelo

**ChatGPT** es un modelo conversacional de OpenAI que, además de pre-training y SFT, ha sido ajustado mediante RLHF.

- ▶ Usan un modelo GPT base (por ejemplo, GPT-3)
- ▶ Aplican SFT
- ▶ Utiliza RLHF para el alineamiento con preferencias humanas
- ▶ Pipeline de alineamiento:
  - ▶ Dataset humano de (instrucción, respuestas ordenadas por calidad)
  - ▶ Entrenamiento de un Reward Model a partir de rankings humanos
  - ▶ Optimización de la política mediante PPO

# Takeaways

- ▶ SFT pueden seguir instrucciones pero tiene limitaciones al no generar respuestas *humanas*
- ▶ Para solucionar este problema se usa reinforcement learning
- ▶ La idea básica es maximizar la probabilidad del modelo a que genere respuestas humanas sin desviarse mucho del modelo original
- ▶ Se usa un dataset etiquetado humano formado por pares (*instrucción, lista de respuestas ordenadas por calidad*)
- ▶ **Importante.**
  - ▶ El pre-training + SFT está muy estandarizado. Las grandes empresas son *relativamente* transparentes en esos procesos
  - ▶ Esto no ocurre con la fase de RL. Las empresas que entrenan estos modelos no dan mucha información sobre esa fase

Parameter efficient fine-tuning (PEFT)

# Problema principal de fine-tuning

## Problema principal

En la fase de fine-tuning, se modifican **todos los parámetros**.

Esto es un problema para la **gente humilde** si queremos fine-tunear un modelo grande:

- ▶ Para cada parámetro hay que almacenar:
  - ▶ El valor del parámetro
  - ▶ Su gradiente
  - ▶ Estados adicionales del optimizador (p. ej., momentos en Adam)
- ▶ El consumo de memoria crece linealmente con el número de parámetros
- ▶ El coste computacional del backward es muy elevado
- ▶ Se necesita una GPU con mucha memoria incluso para tareas sencillas

## Ejemplo: coste de fine-tuning (1B parámetros)

Modelo con:

$$N = 10^9 \text{ parámetros}$$

- ▶ Precisión FP32 + Adam
- ▶ Por parámetro:
  - ▶ Parámetro + gradiente + 2 momentos

$$\Rightarrow 16 \text{ bytes / parámetro}$$

Memoria mínima

$$10^9 \times 16 \text{ bytes} = 16 \text{ GB}$$

En la práctica  $\gg 16 \text{ GB}$

Una tarjeta gráfica NVIDIA de 16-24 GBs ronda 800 - 1.5k euros

# Técnicas PEFT

Existen varias técnicas PEFT para solucionar este problema:

- ▶ **Congelación de parámetros**
- ▶ **LoRA**
- ▶ Prefix tuning
- ▶ ...

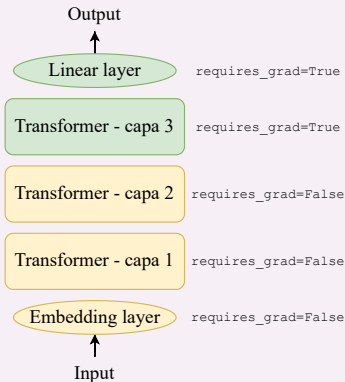
# Congelación de parámetros

## Definición

Consiste en fine-tunear un subconjunto de los parámetros del modelo.

- ▶ Muy popular en visión, se congelan las primeras capas (feature layers) y se fine-tunean las últimas
- ▶ En NLP es similar ya que los encoder y decoder tiene varias *capas/bloques* de transformers (BERT-base tiene 12 capas)
- ▶ Los modelos actuales también se dividen en bloques o capas
- ▶ En PyTorch se puede implementar esto de forma muy sencilla. Lo único que hay que hacer es poner `requires_grad=False` a todas las capas que no quieras fine-tunear

# Congelación de parámetros



- El número de parámetros a optimizar es el número de parámetros que quedan con `requires_grad=True`

## LoRA (Low-Rank Adaptation)

**LoRA** es una técnica de PEFT propuesta por Microsoft en la que se adapta un LLM mediante **actualizaciones de bajo rango** en ciertas matrices, **sin modificar los pesos originales** del modelo.

- ▶ El modelo base permanece congelado
- ▶ En cada capa, se aprende una adaptación de bajo rango
- ▶ La adaptación guía el comportamiento del modelo

# LoRA

Imaginemos que tenemos una red cualquiera y, en alguna parte, se hace la siguiente multiplicación:

$$h = W_o x$$

en este caso  $W_o \in \mathbb{R}^{d \times k}$ . Así pues, esta matriz tendría un total de  $d \times k$  parámetros. Los autores proponen, aplicar algo como:

$$h = (W_o + \Delta W)x$$

dejando  $W_o$  congelado y aprendiendo  $\Delta W$ . Este  $\Delta W$  lo definen como  $\Delta W_o = BA$  donde

$$B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}$$

y  $r$  (usualmente  $= 2, 4, 8$ ) es un hiperparámetro  $\ll \min(d, k)$ . De modo que el número total de parámetros sería  $r(d + k) \ll d \times k$ .

# LoRA

En LoRA:

- ▶ Se congela todo el modelo
- ▶ Se escogen una serie de matrices. Se suelen escoger, de todas las capas:
  - ▶ proyecciones lineales de atención, usualmente ["q\_proj", "v\_proj"]
  - ▶ algunas veces a las capas de feed-forward (no tan común)
- ▶ Se aplica el principio de la diapositiva anterior en cada una de las matrices escogidas

LoRA (o una variante) es la técnica PEFT más utilizada.

# LoRA

En LoRA:

- ▶ Se congela todo el modelo
- ▶ Se escogen una serie de matrices. Se suelen escoger, de todas las capas:
  - ▶ proyecciones lineales de atención, usualmente ["q\_proj", "v\_proj"]
  - ▶ algunas veces a las capas de feed-forward (no tan común)
- ▶ Se aplica el principio de la diapositiva anterior en cada una de las matrices escogidas

LoRA (o una variante) es la técnica PEFT más utilizada.

Demostración en Colab comparando el número de parámetros entrenables

Demostración en Colab con Qwen/Qwen2.5-1.5B

# Cuantización

# Problema principal de la inferencia con LLMs

## Problema principal

En inferencia, los LLMs modernos utilizan **pesos en FP16 o FP32**.

Esto es un problema para la **gente humilde** si queremos **usar** modelos grandes:

- ▶ Los pesos del modelo deben residir en memoria:
  - ▶ GPU (VRAM) para baja latencia
  - ▶ O CPU (RAM) con gran penalización de velocidad
- ▶ Muchos modelos **no caben** en GPUs de consumo
- ▶ Si no cabe la inferencia en GPU, tenemos que irnos a CPU

## Ejemplo: coste de inferencia (7B parámetros)

Modelo con:

$$N = 7 \times 10^9 \text{ parámetros}$$

- ▶ Precisión FP16
- ▶ Por parámetro:
  - ▶ Peso en FP16

$$\Rightarrow 2 \text{ bytes / parámetro}$$

Memoria mínima

$$7 \times 10^9 \times 2 \text{ bytes} = 14 \text{ GB}$$

En la práctica  $\gg$  16 GB de VRAM

Una tarjeta gráfica NVIDIA de 16-24 GBs ronda 800 - 1.5k euros

# Solución: Cuantización

## Idea clave

La Cuantización reduce el consumo de memoria representando los pesos del modelo con **menos bits, sin reentrenar el LLM**.

En lugar de usar:

FP32 / FP16

Usamos:

INT8, INT4, ...

- ▶ El modelo base no se modifica
- ▶ Sólo cambia la representación numérica de los pesos
- ▶ El comportamiento del modelo se mantiene *razonablemente*

## Ejemplo: efecto de la cuantización

Modelo con:

$$N = 7 \times 10^9 \text{ parámetros}$$

► FP16:

$$7B \times 2 \text{ bytes} = 14 \text{ GB}$$

► INT8:

$$7B \times 1 \text{ byte} = 7 \text{ GB}$$

► INT4:

$$7B \times 0.5 \text{ bytes} \approx 3.5 \text{ GB}$$

### Conclusión

La cuantización hace posible ejecutar LLMs grandes en **GPUs de consumo** e incluso en CPU.

## Mensaje final: cuantizar no es hacer casting

### Idea clave

La cuantización no consiste en convertir tipos numéricos, sino en aplicar **técnicas específicas** para reducir precisión **controlando el error**.

Ejemplos de técnicas Post-Training Quantization (PTQ):

- ▶ **PTQ básica (per-tensor / per-channel)**. Cuantiza el modelo usando una única escala global o por canal.
- ▶ **PTQ group-wise (K-quantization)**. Divide los pesos en grupos pequeños y asigna una escala por grupo, reduciendo el error localmente (usado en **Ollama**).
- ▶ **PTQ optimizada (GPTQ, AWQ)**. Minimiza explícitamente el error capa a capa o según activaciones, usando información estructural o datos de calibración.

Librerías para inferencia

# Librerías para interactuar con LLMs

## Idea principal

Los LLMs pueden utilizarse directamente desde distintos entornos y APIs, con **diferentes niveles de control, coste y dependencia externa.**

- ▶ **Librerías open-source** para ejecución local y control total
  - ▶ HuggingFace
  - ▶ Ollama
- ▶ **APIs comerciales:** modelos propietarios vía llamada remota
  - ▶ OpenAI API
  - ▶ Gemini API

Librerías open-source para ejecución local

## Qué es

Ecosistema open-source para trabajar con modelos de lenguaje y visión, tanto en local como en la nube.

- ▶ Repositorio de miles de modelos pre-entrenados con sus tokenizadores
- ▶ Soporte para:
  - ▶ Inferencia
  - ▶ Pre-training
  - ▶ Fine-tuning (incluso post-training)
  - ▶ Evaluación
- ▶ Muy usado en investigación y prototipado

# HuggingFace: Librerías principales

## ▶ Transformers

- ▶ Modelos pre-entrenados para NLP, visión y multimodal
- ▶ APIs unificadas para inferencia y fine-tuning
- ▶ Arquitecturas como BERT, GPT, T5, ViT, etc.

## ▶ Datasets

- ▶ Carga y procesamiento eficiente de datasets
- ▶ Soporte para streaming y grandes volúmenes de datos
- ▶ Integración directa con Transformers

## ▶ Tokenizers

- ▶ Tokenización rápida
- ▶ BPE, WordPiece, Unigram, etc.
- ▶ Consistente con los modelos publicados

## ▶ PEFT

- ▶ Fine-tuning eficiente (LoRA, adapters, prefix tuning)
- ▶ Reduce coste computacional y memoria

## ▶ TRL

- ▶ Entrenamiento con feedback humano
- ▶ Implementaciones de RLHF, PPO, DPO

# HuggingFace: ejemplo inferencia

## Modo bajo nivel

1. Carga el tokenizador y modelo (solo una vez)
2. Formatea tu prompt de manera adecuada siguiendo la documentación
3. Tokeniza y codifica tu prompt
4. Genera la salida
5. Decodifica la salida

## Modo alto nivel

Usando la clase *pipeline*.

Ejemplo en el notebook/colab.

# Ollama

## Idea principal

**Ollama** es una herramienta para **ejecutar modelos de lenguaje grandes de forma local**, simplificando su descarga, gestión y ejecución mediante una interfaz unificada.

- ▶ Permite ejecutar LLMs **en local** (CPU o GPU)
- ▶ Pensado para desarrolladores y experimentación rápida
- ▶ Interfaz sencilla tipo `docker run` para modelos

# Modelos soportados por Ollama

- ▶ Modelos generales: LLaMA, Mistral, Qwen
- ▶ Modelos orientados a código: CodeLLaMA, DeepSeek-Coder
- ▶ Modelos instruct (chat / instruct)
- ▶ Modelos cuantizados (4-bit, 8-bit)
- ▶ Lista de modelos: <https://ollama.com/search>

## Nota

Ollama abstrae detalles como cuantización, carga en GPU/CPU y formatos internos del modelo.

# Uso básico de Ollama

## Descargar y ejecutar un modelo

```
ollama pull qwen2.5:0.5b
```

```
ollama run qwen2.5:0.5b
```

- ▶ Descarga automática del modelo
- ▶ Ejecución interactiva en terminal
- ▶ Persistencia del modelo en local

# Ollama como servidor de inferencia

- ▶ Ollama expone una **API HTTP**
- ▶ Puede usarse desde Python, JavaScript, etc.
- ▶ Ideal para integrarlo en aplicaciones o agentes

```
import ollama

response = ollama.chat(
    model="qwen2.5:0.5b",
    messages=[
        {"role": "system", "content": "You are a helpful assistant"},
        {"role": "user", "content": "Write a Python function for binary search"}
    ],
    options={
        "temperature": 0.3
    }
)

print(response["message"]["content"])
```

# Ollama ejemplo con Docker

- ▶ Ejemplo descarga y ejecución desde CLI
- ▶ Conexión con OpenWeb UI

```
docker run -d \  
  --name openwebui \  
  -p 3000:8080 \  
  -e OLLAMA_BASE_URL=http://host.docker.internal:11434 \  
  --restart unless-stopped \  
  ghcr.io/open-webui/open-webui:main
```

APIs comerciales

# OpenAI y Gemini APIs

## Características principales

APIs comerciales que da acceso a modelos GPT y Gemini de última generación para tareas de generación de texto.

- ▶ Modelos de propósito general y especializados (texto, código, razonamiento)
- ▶ Control de generación: temperature, max tokens, etc.
- ▶ Amplio ecosistema de SDKs (Python, JavaScript, etc.)

## Ventajas y limitaciones

- ▶ **Ventajas:** alta calidad, robustez, buena alineación con instrucciones
- ▶ **Limitaciones:** coste, dependencia del proveedor, modelos cerrados

## Ejemplo: OpenAI API (Python)

```
pip install openai

from openai import OpenAI

client = OpenAI(api_key="YOUR_API_KEY")

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Explain quicksort and give Python code."}
    ],
    temperature=0.2,
    max_tokens=300
)

print(response.choices[0].message.content)
```

## Ejemplo: Gemini API (Python)

```
pip install -q -U google-genai
from google import genai
import os

os.environ["GEMINI_API_KEY"] = "YOUR_API_KEY"
client = genai.Client()

response = client.models.generate_content(
    model="gemini-2.5-flash", contents="Explain quicksort and give Python code."
)
print(response.text)
```

# Ejemplos

Vamos a ver ejemplos en los notebooks:

- ▶ Ollama local
- ▶ Ollama cloud
- ▶ Gemini
- ▶ OpenAI
- ▶ Ejercicio feedback loop

# Agentes LLM

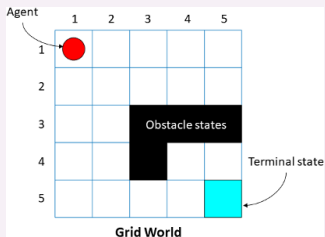
# Agentes IA

## Definición de agentes IA

Los agentes IA son entidades artificiales que pueden percibir y actuar de forma autónoma sobre el entorno circundante para lograr objetivos específicos.

- ▶ Los agentes IA no es algo nuevo
- ▶ Los antiguos agentes funcionaban con reinforcement learning
- ▶ Agentes poco flexibles, estados y acciones predefinidas

**Ejemplo.** Un agente que aprende a moverse en una cuadrícula para alcanzar un objetivo evitando obstáculos.



# Agentes LLM

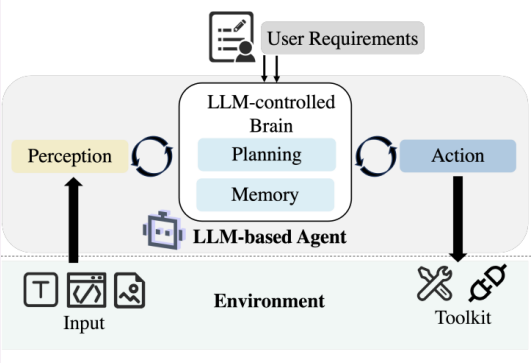
## Definición de agentes LLM

Son agentes que usan LLMs como el cerebro. Se puede ver como una forma de extender los LLMs permitiéndoles **interactuar con herramientas y entornos externos**.

Tienen tres componentes:

- ▶ **Plan:** define la estrategia que sigue el agente para alcanzar un objetivo, descomponiendo una tarea compleja en subtareas manejables
- ▶ **Memoria:** permite al agente mantener contexto, aprender de interacciones previas y evitar razonamientos redundantes
- ▶ **Percepción y acción:** este componente conecta el razonamiento del LLM con el entorno. Usualmente a través de herramientas que interactúan con el entorno

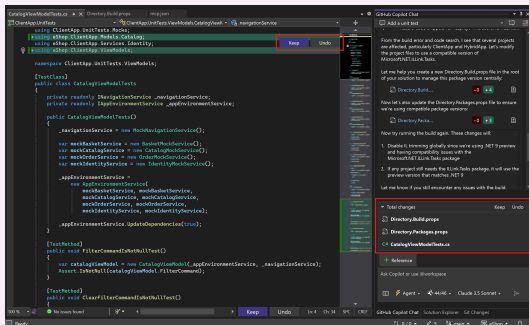
# Agentes LLM



# Agentes LLM

Un ejemplo es el agente de GitHub Copilot:

- ▶ Usa un LLM como motor principal (se puede seleccionar el LLM)
- ▶ Percibe el contexto: ficheros del proyecto actual e incluso puede estar conectado con herramientas externas
- ▶ Actúa sugiriendo líneas, funciones completas o tests



Patrones *agénticos*

## Definición

Los patrones *agénticos* organizan cómo interactúan esos tres componentes (plan, memoria y percepción/acción).

Vamos a ver los más usados:

- ▶ ReAct: el LLM genera trazas de *thought*, *action* y *observation*
- ▶ CodeAct: es como ReAct, pero las acciones son llamadas programáticas a APIs
- ▶ Planning first: primero se genera un plan completo explícito y se ejecuta poco a poco

ReAct

# ReAct

ReAct es una estrategia muy sencilla e intuitiva. Dada una *user request*, el sistema sigue un bucle que genera:

- ▶ **Thought:** razonamiento interno
- ▶ **Action:** llamada a una herramienta
- ▶ **Observation:** resultado devuelto por la herramienta

hasta que la respuesta final es obtenida.

(1d) ReAct (Reason + Act)

```
Thought 1: I need to search Apple Remote and find the
program it was originally designed to interact with.
Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced in
October 2005 by Apple ... originally designed to control the
Front Row media center program ...

Thought 2: Apple Remote was originally designed to control
the Front Row media center program. I need to search Front
Row next and find what other device can control it.
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ['Front Row
Seat to Earth', 'Front Row Motorsports', 'Front Row
(software)', ...]

Thought 3: Front Row is not found. I need to search Front
Row (software) .
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple
Remote or the keyboard function keys. So the answer is
keyboard function keys.
Act 4: Finish[keyboard function keys]
```



# ReAct

```
system_prompt = """
You are an intelligent agent that can reason and act.

You have access to the following tools:
{tool_descriptions}

Use the following format:

Question: the question to be answered
Thought: your reasoning about what to do next
Action: the action to take, must be one of [{tool_names}]
Action Input: the input to the action
Observation: the result of the action
... (this Thought/Action/Observation can repeat multiple times)
Thought: I now know the final answer
Final Answer: the answer to the original question
"""

user_prompt = "Question: {user_request}"
```

Un patrón común es forzar que las llamadas a herramientas se indiquen con JSON.

## ReAct: ejemplo

**User request.** Who is the president of the USA today?

**Tools.** web\_search

Possible flow:

1. **Thought:** This can change over time, so I should check an authoritative source.

## ReAct: ejemplo

**User request.** Who is the president of the USA today?

**Tools.** web\_search

Possible flow:

1. **Thought:** This can change over time, so I should check an authoritative source.
2. **Action:** {"tool": "web\_search", "args": {"query": "president of the USA"}}  
Aquí la generación del LLM se para, el sistema parsea la acción (e.g., con regex) y se invoca la herramienta. El resultado se copia en observacion.

## ReAct: ejemplo

**User request.** Who is the president of the USA today?

**Tools.** web\_search

Possible flow:

1. **Thought:** This can change over time, so I should check an authoritative source.
2. **Action:** {"tool": "web\_search", "args": {"query": "president of the USA"}}  
Aquí la generación del LLM se para, el sistema parsea la acción (e.g., con regex) y se invoca la herramienta. El resultado se copia en observacion.
3. **Observation:** Donald J. Trump is the President of the United States.

## ReAct: ejemplo

**User request.** Who is the president of the USA today?

**Tools.** web\_search

Possible flow:

1. **Thought:** This can change over time, so I should check an authoritative source.
2. **Action:** {"tool": "web\_search", "args": {"query": "president of the USA"}}  
Aquí la generación del LLM se para, el sistema parsea la acción (e.g., con regex) y se invoca la herramienta. El resultado se copia en observacion.
3. **Observation:** Donald J. Trump is the President of the United States.
4. **Thought:** I now know the final answer.

## ReAct: ejemplo

**User request.** Who is the president of the USA today?

**Tools.** web\_search

Possible flow:

1. **Thought:** This can change over time, so I should check an authoritative source.
2. **Action:** {"tool": "web\_search", "args": {"query": "president of the USA"}}  
Aquí la generación del LLM se para, el sistema parsea la acción (e.g., con regex) y se invoca la herramienta. El resultado se copia en observacion.
3. **Observation:** Donald J. Trump is the President of the United States.
4. **Thought:** I now know the final answer.
5. **Final Answer:** The current president of the United States is Donald J. Trump.  
Se detecta que el LLM ha llegado a la Final Answer y se para la generación.

CodeAct

## Idea principal

**CodeAct** es un patrón agéntico similar a ReAct. La diferencia es que, en la etapa de acción, se genera código y las herramientas son llamadas a APIs.

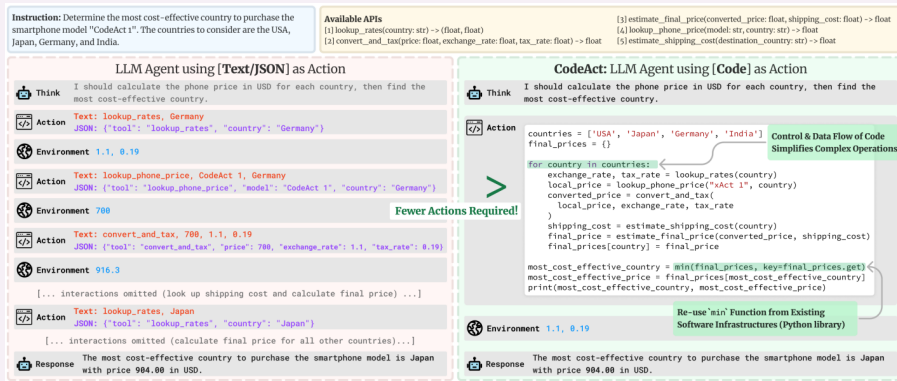


Figura: Comparación con ReAct.

## CodeAct: ejemplo

**User request.** Determine the most cost-effective country to purchase the smartphone model iPhone 11

**Available APIs.** `lookup_rates`, `lookup_phone_price`, `convert_and_tax`, `estimate_shipping_cost`, `estimate_final_price`

Possible flow (CodeAct):

1. **Thought:** I should calculate the phone price in USD for each country, then find the most cost-effective country.

## CodeAct: ejemplo

**User request.** Determine the most cost-effective country to purchase the smartphone model iPhone 11

**Available APIs.** lookup\_rates, lookup\_phone\_price, convert\_and\_tax, estimate\_shipping\_cost, estimate\_final\_price

Possible flow (CodeAct):

1. **Thought:** I should calculate the phone price in USD for each country, then find the most cost-effective country.

2. **Action:**

```
countries = ['USA', 'Japan', 'Germany', 'India']
final_prices = {}
for country in countries:
    ... # calcular el precio teniendo en cuenta el tax, exchange, etc.
best_country = min(final_prices, key=final_prices.get)
print(best_country, final_prices[best_country])
```

La generación se para y se ejecuta el código, se añade el resultado del print en la observation.

## CodeAct: ejemplo

**User request.** Determine the most cost-effective country to purchase the smartphone model iPhone 11

**Available APIs.** lookup\_rates, lookup\_phone\_price, convert\_and\_tax, estimate\_shipping\_cost, estimate\_final\_price

Possible flow (CodeAct):

1. **Thought:** I should calculate the phone price in USD for each country, then find the most cost-effective country.

2. **Action:**

```
countries = ['USA', 'Japan', 'Germany', 'India']
final_prices = {}
for country in countries:
    ... # calcular el precio teniendo en cuenta el tax, exchange, etc.
best_country = min(final_prices, key=final_prices.get)
print(best_country, final_prices[best_country])
```

La generación se para y se ejecuta el código, se añade el resultado del print en la observation.

3. **Observation:** Japan 904.00

## CodeAct: ejemplo

**User request.** Determine the most cost-effective country to purchase the smartphone model iPhone 11

**Available APIs.** lookup\_rates, lookup\_phone\_price, convert\_and\_tax, estimate\_shipping\_cost, estimate\_final\_price

Possible flow (CodeAct):

1. **Thought:** I should calculate the phone price in USD for each country, then find the most cost-effective country.

2. **Action:**

```
countries = ['USA', 'Japan', 'Germany', 'India']
final_prices = {}
for country in countries:
    ... # calcular el precio teniendo en cuenta el tax, exchange, etc.
best_country = min(final_prices, key=final_prices.get)
print(best_country, final_prices[best_country])
```

La generación se para y se ejecuta el código, se añade el resultado del print en la observation.

3. **Observation:** Japan 904.00
4. **Final Answer:** The most cost-effective country is Japan with price 904.00 in USD.

Planning first

# Planning first

## Idea principal

El agente genera **un plan explícito antes de actuar** (descomposición de la tarea en pasos) y lo ejecuta paso a paso. En algunas variantes, el plan puede revisarse durante la ejecución.

Existen múltiples arquitecturas basadas en *planning-first*:

- ▶ **Planning + ReAct / CodeAct**: el agente genera un plan global y ejecuta cada paso mediante razonamiento y acción iterativos
- ▶ **Planning + ReAct / CodeAct + Replanning**: el agente puede **modificar el plan** tras observar resultados inesperados
- ▶ **Planner–Executor**: un agente planificador genera los pasos y uno o varios agentes ejecutores (ReAct/CodeAct) los llevan a cabo
- ▶ Muchas más variantes

## Planning first: ejemplo

**User request.** Implementar una función `is_palindrome(s: str) -> bool` en Python y asegurar que pasa todos los tests.

**Herramientas.** Herramientas para escribir y ejecutar código

Posible flow:

- ▶ **Planning:** I need to implement a function `is_palindrome`. Plan:

- ▶ Implement a simple version.
- ▶ Run the tests.
- ▶ If tests fail, analyze the error and update the plan.

- ▶ Secuencia de thought-action-observation que da lugar a:

```
def is_palindrome(s: str) -> bool:  
    s = s.lower().replace(" ", "")  
    return s == s[::-1]
```

- ▶ Se ejecutan los tests:

```
AssertionError: test failed for input  
"¿Acaso hubo búhos acá?"
```

## Planning first: ejemplo

- ▶ **Replanning.** The original plan assumed only spaces and case differences. This observation reveals missing requirements:

- ▶ punctuation must be ignored
- ▶ Unicode accents must be normalized

I need to update the plan before continuing.

- ▶ Normalize Unicode characters.
- ▶ Remove non-alphanumeric symbols.
- ▶ Reimplement `is_palindrome`.

- ▶ Secuencia de thought-action-observation que da lugar a:

```
import unicodedata
import re

def is_palindrome(s: str) -> bool:
    s = unicodedata.normalize("NFD", s)
    s = "".join(c for c in s if unicodedata.category(c) != "Mn")
    s = re.sub(r"[^a-zA-Z0-9]", "", s.lower())
    return s == s[::-1]
```

- ▶ Se ejecutan los tests:

All tests passed

Librerías de agentes

# Librerías de agentes

## Librerías de agentes

Son librerías que permiten programar **agentes inteligentes** sin preocuparse por los detalles técnicos de bajo nivel (parsear mensajes, extraer las acciones, etc.).

- ▶ Predominantemente desarrolladas como **librerías Python**
- ▶ Tecnologías **emergentes**, con APIs aún inestables y evolución continua
- ▶ Ejemplos: **smolagents**, **LangChain**, LangGraph, AutoGen, etc.

Smolagents

# Smolagents

## Smolagents

smolagents es una librería open-source de HuggingFace. Es una librería ligera para construir **agentes LLM** muy centrada en CodeAct.

- ▶ Enfoque **minimalista** con pocas abstracciones
- ▶ Soporta **tool calling** (herramientas tipadas/descritas)
- ▶ Patrones típicos: **ReAct**, **Planning+Act**, **CodeAct**
- ▶ Integra bien con ecosistema HF (modelos, spaces, etc.)

# Smolagents

Definición de herramientas. La documentación es importante.

```
from smolagents import tool
```

```
@tool
```

```
def run_sql_query(query: str) -> str:
```

```
    """
```

```
    Allows you to perform SQL queries on the table.
```

```
    Returns a string representation of the result.
```

```
    Args:
```

```
        query: The query to perform. This should be correct SQL.
```

```
    """
```

```
    # lógica para acceder a la base de datos y ejecutar la query
```

```
    # [...]
```

```
    return result
```

```
@tool
```

```
def get_schema() -> str:
```

```
    """
```

```
    Returns a string representation of the database schema.
```

```
    """
```

```
    # lógica para obtener el esquema
```

```
    # [...]
```

```
    return result
```

# Smolagents

CodeAgent:

```
from smolagents import CodeAgent, HfApiModel

agent = CodeAgent(
    tools=[run_sql_query, get_schema],
    model=HfApiModel("meta-llama/Meta-Llama-3.1-8B-Instruct"),
)
agent.run("Can you give me the name of the client who got the most expensive receipt?")
```

Agent ReAct:

```
from smolagents import ToolCallingAgent, HfApiModel

agent = ToolCallingAgent(
    tools=[run_sql_query, get_schema],
    model=HfApiModel("meta-llama/Meta-Llama-3.1-8B-Instruct"),
)
agent.run("Can you give me the name of the client who got the most expensive receipt?")
```

# Smolagents

El planning se controla con el argumento `planning_interval`.

```
from smolagents import ToolCallingAgent, HfApiModel

agent = ToolCallingAgent(
    tools=[run_sql_query, get_schema],
    model=HfApiModel("meta-llama/Meta-Llama-3.1-8B-Instruct"),
    planning_interval=3
)
agent.run("Can you give me the name of the client who got the most expensive receipt?")
```

# Smolagents: ejercicio

Vamos al colab/notebook:

- ▶ Smolagents + Ollama local
- ▶ Smolagents + Ollama cloud
- ▶ Smolagents + Gemini
- ▶ Smolagents + OpenAI
- ▶ Ejercicio: asistente SQL

LangChain

# LangChain

## LangChain

LangChain es un framework open-source diseñado para simplificar el desarrollo de Agentes LLMs.

- ▶ Facilita la integración de LLMs con fuentes de datos externas, herramientas y memoria
- ▶ Soporta múltiples proveedores de LLMs (OpenAI, Anthropic, Google, etc.)
- ▶ Ecosistema completo: **LangChain** (cadenas y agentes simples), LangGraph (agentes avanzados), LangSmith (observabilidad/logging), LangServe (despliegue)



# Bloques principales de LangChain

LangChain se estructura en varios componentes clave:

- ▶ **Models**: wrappers sobre LLMs (OpenAI, Ollama, HuggingFace, etc.)
- ▶ Prompts: plantillas parametrizables y reutilizables
- ▶ Chains: secuencias de llamadas (Lang**Chain**)
- ▶ Tools: herramientas
- ▶ Memory: gestión de contexto a corto y largo plazo
- ▶ **Agents**: agentes básicos ya implementados

# Agentes

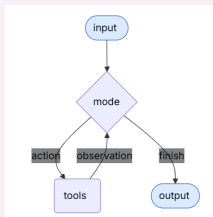
- ▶ LangChain ofrece `create_agent` para crear un agente ReAct
- ▶ Fácil de usar: una sola función para crear un agente con herramientas
- ▶ Soporte para middleware (e.g, human-in-the-loop, etc.)

```
@tool("square_root")
def tool1(x: float) -> float:
    """Calculate the square root of a number"""
    return x ** 0.5

agent = create_agent(
    model="gpt-5-nano",
    tools=[tool1],
    system_prompt="You are an arithmetic wizard." +
    "Use your tools to calculate the square root and square of any number."
)
```

# Agentes

- ▶ `create_agent` del LangChain funciona bien cuando el modelo soporta ***tool calling***
- ▶ Un modelo soporta tool calling cuando ha sido fine-tuneado con un dataset especial de *tool calling*<sup>1</sup>
- ▶ En esos modelos el *thought*, o bien, 1) ocurre *internamente* y no se expone, 2) o no ocurre. Depende de como haya sido fine-tuneado
- ▶ Estos modelos devuelven *tool calls* que deben ser ejecutadas



Ejercicio en Colab

<sup>1</sup><https://huggingface.co/datasets/Jofthomas/hermes-function-calling-thinking-V1>

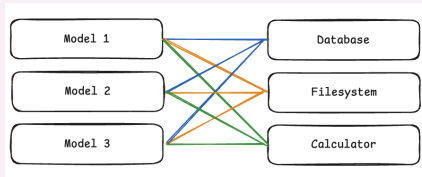
# Model Context Protocol

## El problema $M \times N$

- ▶ Las aplicaciones y frameworks basados en LLMs están creciendo a un ritmo acelerado
- ▶ Para que estas aplicaciones sean realmente útiles, deben interactuar con herramientas externas y fuentes de datos

### Problema $M \times N$

El problema de integración  $M \times N$  describe la dificultad de conectar  $M$  aplicaciones basadas en LLMs con  $N$  herramientas externas o fuentes de datos **en ausencia de un mecanismo de integración estandarizado**.



**Figura:** Problema  $M \times N$  extraído de <https://huggingface.co/learn/mcp-course/unit1/key-concepts>

# Model Context Protocol

- ▶ El Model Context Protocol (MCP) surge para resolver este problema
- ▶ Propuesto por Anthropic
- ▶ Se trata de una interfaz estándar
- ▶ Cada aplicación LLM implementa el lado del cliente sólo una vez
- ▶ Cada aplicación/herramienta externa implementa el lado del servidor sólo una vez
- ▶ Pasamos de  $M \times N$  a  $M + N$

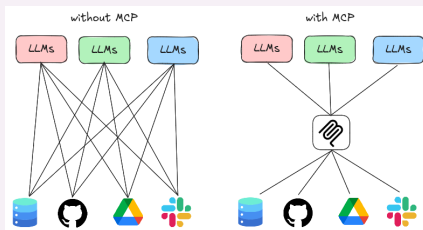


Figura: Extraído de <https://hoangndst.com/blog/model-context-protocol>

# Arquitectura MCP

La arquitectura está formada por tres componentes:

- ▶ **Host**: aplicación LLM orientada al usuario final, con la que los usuarios interactúan directamente
- ▶ **Client**: componente dentro del Host que gestiona la comunicación con un servidor MCP específico
- ▶ **Server**: programa o servicio externo que expone capacidades a los modelos LLM a través del protocolo MCP

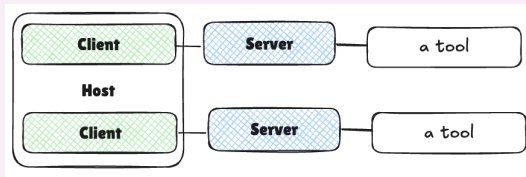


Figura: Extraída de

<https://huggingface.co/learn/mcp-course/unit1/architectural-components>

# Lo que exponen los servers

Los servers pueden exponer:

- ▶ **Tools.** Herramientas o funciones que los LLMs pueden llamar.
- ▶ **Resources.** Fuentes de datos pasivas que proporcionan acceso de solo lectura: el contenido de archivos, esquemas de bases de datos o documentación de APIs
- ▶ **Prompts.** Plantillas de instrucciones predefinidas que indican al modelo cómo trabajar con herramientas y recursos específicos

**Nota.** Los resources y los prompts no los usa el LLM de manera autónoma. Son gestionados por el usuario y la aplicación. Por ejemplo, en el GitHub Copilot de VSCode se añaden en el contexto solo si se lo indica el usuario

**Buen ejemplo.**

[https://github.com/GuangYiDing/mysql-mcp/blob/main/mysql\\_mcp.py](https://github.com/GuangYiDing/mysql-mcp/blob/main/mysql_mcp.py)

# Protocolo de comunicación: mensajes

- ▶ Los mensajes se escriben usando el formato JSON-RPC 2.0
- ▶ Tres tipos de mensajes:
  - ▶ Requests. Cliente al servidor, para iniciar una operación. Por ejemplo, una llamada a una herramienta

```
{  
  "jsonrpc": "2.0",  
  "id": 1,  
  "method": "tools/call",  
  "params": {  
    "name": "weather",  
    "arguments": {  
      "location": "San Francisco"  
    }  
  }  
}
```

- ▶ Responses. Servidor al cliente. Por ejemplo, resultado de haber ejecutado una herramienta
  - ▶ Notifications. Usualmente del servidor al cliente para proporcionar actualizaciones o notificaciones sobre eventos

# Protocolo de comunicación: mecanismos de transporte

Hay dos posibles mecanismos de transporte:

- ▶ **stdio (Standard Input/Output)**

- ▶ Cuando el cliente y el servidor están en la misma máquina
- ▶ La aplicación Host lanza el Server como un subproceso y se comunica con él escribiendo en su entrada estándar (stdin) y leyendo desde su salida estándar (stdout)

- ▶ **HTTP + SSE (Server-Sent Events)**

- ▶ Cuando el Client y el Server se ejecutan en máquinas diferentes
- ▶ La comunicación se realiza sobre HTTP
- ▶ El servidor envía actualizaciones continuas mediante Server-Sent Events (SSE)

# Protocolo de comunicación: ciclo de vida

## 1. Inicialización

El Client se conecta al Server e intercambian versión del protocolo y capacidades soportadas.

## 2. Descubrimiento

El Client solicita las capacidades disponibles (tool, recursos y prompts). El Server responde con las capacidades expuestas.

## 3. Ejecución

1. El cliente solicita la ejecución de una herramienta
2. (Opcional) El servidor envía notificación del progreso
3. El servidor envía respuesta final

## 4. Terminación

La conexión se cierra de forma ordenada.

# MCP SDKs

- Hay MCP SDKs en TypeScript, Python, Java, Kotlin, etc.

```
mcp = FastMCP("Weather Service")
```

```
@mcp.tool()
def get_weather(location: str) -> str:
    """Get the current weather for a specified location."""
    return f"Weather in {location}: Sunny, 72°F"

@mcp.resource("weather://{location}")
def weather_resource(location: str) -> str:
    """Provide weather data as a resource."""
    return f"Weather data for {location}: Sunny, 72°F"

@mcp.prompt()
def weather_report(location: str) -> str:
    """Create a weather report prompt."""
    return f"""\nYou are a weather reporter. Weather report for {location}?\n"""

if __name__ == "__main__":
    # studio
    mcp.run()
```

- Solo hay que iniciar el server y conectarlo a las aplicaciones (GitHub Copilot VSCode, Claude Desktop App, etc.)
- Clientes: <https://modelcontextprotocol.io/clients>